persistence layer. If you'd like to explore how to model that scenario, I recommend referring to domain-driven design and hexagonal architecture principles.

In this case, the business domain is quite simple, so we'll update the `Book` record to also be a persistent entity.

MAKING A DOMAIN CLASS PERSISTENT

Spring Data JDBC encourages working with immutable entities. Using Java records to model entities is an excellent choice, since they're immutable by design and expose an all-args constructor that the framework can use to populate objects.

A persistent entity must have a field that acts as the identifier for the object, which will translate to the primary key in the database. You can mark a field as an identifier with the `@Id` annotation (from the `org.springframework.data.annotation` package). The database is responsible for generating a unique identifier for each created object automatically.

NOTE Books are uniquely identified by an ISBN, which we can call a *natural key* (or *business key*) for the domain entity. We could decide to use it also as the primary key or introduce a *technical key* (or *surrogate key*). There are pros and cons to both approaches. I chose to use a technical key to make it easier to manage and to decouple domain concerns from persistence implementation details.

That is enough to create and persist a `Book` in the database. It's also OK when a single user updates an existing `Book` object in isolation. But what happens if the same entity is updated by multiple users concurrently? Spring Data JDBC supports *optimistic locking* to address that concern. Users can read data concurrently. When a user attempts an update operation, the application checks if there has been any change since the last read. If there